

White-Label Super App — Backend Engineering Spec

Sep 30, 2026 · @Abbas Alayan

This spec tells backend engineers exactly what to build: the chosen stack, every service, every database table with its columns, every endpoint, the socket events and the messaging topics. It complements the architecture document and uses the same white-label, multi-tenant model: every table, request, event and socket room is scoped by `tenant_id`.

1. Technology Stack

TypeScript on NestJS with PostgreSQL is the default for every service. Kafka carries events, Temporal runs long money workflows, Redis handles cache and sockets, and Go is reserved for the two latency-critical paths.

Layer	Choice	Why
Language	TypeScript (Node.js active LTS)	One language across backend, admin web and tooling; large hiring pool
Hot paths	Go (latest stable) for ledger posting and card authorization	Predictable sub-10 ms latency and low GC pauses where card networks impose hard timeouts
API framework	NestJS on the Fastify adapter	Modular DI, guards, interceptors, OpenAPI generation, WebSocket and microservice support built in
Internal RPC	gRPC (protobuf, buf for schema lint and breaking-change checks)	Typed, fast service-to-service commands
Primary database	PostgreSQL (latest stable) with PostGIS, pgcrypto, pg_partman	ACID for money, row-level security for tenancy, JSONB for config, geo for riders

Layer	Choice	Why
Data access	Prisma for CRUD services; Kysely (typed SQL) for ledger, reporting and heavy queries	Productivity where it helps, full SQL control where correctness and speed matter
Connection pooling	PgBouncer (transaction mode)	Keeps Postgres connections bounded under many pods
Cache, locks, rate limits	Redis (or Valkey), cluster mode	Sub-ms reads, distributed locks, sliding-window limits, GEO for rider positions, Socket.IO adapter
Event streaming	Apache Kafka (managed, e.g. MSK or Confluent) + Schema Registry (Avro)	Durable, ordered, replayable domain events between services
Change data capture	Debezium	Outbox relay and warehouse feed without dual writes
Workflow orchestration	Temporal	Payment sagas, payouts, KYC reviews, disputes and card issuance with retries, timers and audit history
Background jobs	BullMQ on Redis	Short jobs: notifications, exports, webhooks delivery, scheduled tasks
Realtime	Socket.IO on a NestJS gateway service with the Redis adapter	Rooms, acks, reconnection and fallbacks; scales horizontally
Push, SMS, email	FCM (Android and iOS via APNs), SMS and email through tenant <code>MessagingAdapter</code> s	Tenant can bring local SMS providers
Search	OpenSearch	Admin search across users, transactions, merchants, logs
Analytics warehouse	ClickHouse, fed by Debezium CDC	Fast reports and dashboards without touching production
Object storage	S3-compatible with SSE-KMS	KYC documents, statements, exports, media

Layer	Choice	Why
Secrets and keys	HashiCorp Vault + cloud KMS (HSM-backed for card and signing keys)	Per-tenant secret paths and keys
API gateway	Kong Gateway (or Envoy Gateway)	Tenant resolution, JWT validation, WAF plugins, rate limits, request logging
Identity	Custom auth service for customers (phone OTP, PIN, device binding, EdDSA JWTs); Keycloak for staff SSO/OIDC + TOTP	Customer auth needs fintech-specific flows; staff need enterprise SSO
Service mesh	Linkerd (or Istio)	mTLS between services, retries, traffic split for canaries
Runtime	Kubernetes (EKS or GKE), Helm, Terraform, Argo CD	Same deployment for shared and dedicated tenants
Monorepo	Nx with pnpm	Shared libraries, affected-only builds and tests
Observability	OpenTelemetry → Prometheus, Grafana, Loki, Tempo; Sentry for errors	One trace from app tap to ledger posting
Testing	Vitest, Testcontainers, Pact, k6	Real Postgres/Kafka in integration tests, contract tests, load tests
API contracts	OpenAPI 3.1 + AsyncAPI 3	Generated Dart (Flutter) and TypeScript (Next.js) clients; documented socket and Kafka events

Use the latest stable version of every package at project start, pin exact versions in the lockfile, and upgrade through Renovate.

2. Service Map and Repository Structure

The platform is 20 deployable services. Each one owns exactly one Postgres schema, and none reads another service's tables. Inside the cluster every service listens on HTTP 3000 , gRPC 50051 and metrics 9464 .

Service	Language	Owns schema	Exposes	Main responsibility
auth-service	TS	identity	REST, gRPC	Registration, OTP, PIN, devices, sessions, JWT issuing, step-up
tenant-service	TS	tenancy	REST (console), gRPC	Tenant profiles, adapters registry, domains, deployments, app config per tenant
kyc-service	TS	kyc	REST, gRPC, Temporal worker	KYC/KYB applications, documents, provider checks, review cases, tiers
risk-service	TS	risk	gRPC, Kafka consumer	Real-time risk decisions, rules engine, AML alerts and cases, screening
ledger-service	Go	ledger	gRPC only	Accounts, journal entries, holds, balances, FX, reconciliation checks
wallet-service	TS	wallet	REST, gRPC	Wallets, limits and usage, beneficiaries, statements
payments-service	TS	payments	REST, gRPC, Temporal worker	P2P, requests, QR, top-ups, withdrawals, bill pay orchestration, fees

Service	Language	Owns schema	Exposes	Main responsibility
banking-service	TS	banking	gRPC, webhooks, Temporal worker	Bank adapters, linked accounts, transfers, statements, reconciliation
cards-service	TS + Go (auth path)	cards	REST, gRPC, webhooks	Card lifecycle, controls, JIT authorization, clearing, disputes, tokens
bills-service	TS	bills	REST, gRPC	Biller catalog, inquiry, payment via biller adapters
marketplace-service	TS	marketplace	REST, gRPC	Merchants, stores, catalog, carts, orders, promotions, commissions
delivery-service	TS	delivery	REST, gRPC, WS producer	Riders, dispatch, offers, tracking, earnings, COD
payouts-service	TS	payouts	gRPC, Temporal worker	Merchant and rider payout batches, reserves, statements
platform-service	TS	platform	REST, gRPC	Feature flags, SDUI screens, app versions, maintenance, content
notification-service	TS	notify	Kafka consumer, BullMQ	Push, SMS, email, in-app inbox, templates per tenant and language
realtime-gateway	TS	none (Redis)	Socket.IO	Authenticated sockets for apps, merchant, rider, admin

Service	Language	Owns schema	Exposes	Main responsibility
support-service	TS	support	REST	Tickets, messages, macros, escalations
reporting-service	TS	reporting + ClickHouse	REST, BullMQ	Regulatory and finance reports, scheduled exports
audit-service	TS	audit	Kafka consumer, REST (read)	Immutable admin audit log (WORM), approvals
file-service	TS	files	REST, gRPC	Pre-signed uploads, virus scan, file metadata, signed downloads

The tenant admin portal and platform console are Next.js apps that call a thin **admin-bff** and **console-bff** (NestJS). These enforce staff auth, RBAC and maker-checker, then call services over gRPC.

Monorepo layout

```

super-app/
  apps/
    auth-service/          src/{modules,grpc,http,workers}, prisma/, test/
    tenant-service/
    kyc-service/
    risk-service/
    ledger-service/        Go module: cmd/,
internal/{posting,holds,fx,recon}, migrations/
  wallet-service/
  payments-service/
  banking-service/
  cards-service/          + cards-auth/ (Go) for the authorization endpoint
  bills-service/
  marketplace-service/
  delivery-service/
  payouts-service/
  platform-service/
  notification-service/
  realtime-gateway/

```

```

support-service/
reporting-service/
audit-service/
file-service/
admin-bff/
console-bff/
admin-web/           Next.js tenant admin portal
console-web/         Next.js platform console
libs/
  common/            errors, money, ids, pagination, tenant context
  auth/              guards, JWT verify, step-up, RBAC decorators
  db/                Prisma/Kysely helpers, RLS session setter, outbox
  kafka/             producer, consumer, schema registry, idempotent
handler
  temporal/          client, shared activities
  adapters/          bank/, card-issuer/, kyc/, acquirer/, biller/,
messaging/ (interfaces + implementations + simulators)
  proto/             gRPC .proto files (buf)
  contracts/         OpenAPI + AsyncAPI specs, generated clients
  testing/           Testcontainers fixtures, factories
infra/
  terraform/         modules + one workspace per tenant deployment
  helm/              charts + values/<tenant>.yaml
  argocd/
tools/               codegen, migration runner, seed data

```

3. Database Conventions

Every table follows the same rules, so the table lists in sections 4–8 show only each table's own columns.

Standard columns (present on every table unless noted)

Column	Type	Rule
<code>id</code>	<code>uuid</code>	Primary key, UUIDv7 generated in the application
<code>tenant_id</code>	<code>uuid</code>	Not null, FK to <code>tenancy.tenants</code> ; first column of every composite index
<code>created_at</code>	<code>timestampz</code>	Default <code>now()</code>

Column	Type	Rule
<code>updated_at</code>	<code>timestampz</code>	Set by trigger; omitted on append-only tables
<code>version</code>	<code>int</code>	Optimistic locking on mutable tables

Types and naming

- snake_case tables and columns; plural table names; one Postgres schema per service.
- Money: `amount_minor bigint` + `currency char(3)`; never `numeric` floats in application code.
- Status fields: Postgres `text` with a `CHECK` constraint listing allowed values (easier to migrate than enums).
- Flexible data: `jsonb` only for configuration and provider payloads, never for fields you filter on.
- PII: encrypted at the application layer into `bytea` columns with a `_enc` suffix, plus a `_hash` column (HMAC-SHA256, per-tenant key) for exact-match lookup.
- Soft delete: `deleted_at timestampz` only on master data (merchants, products, beneficiaries); never on money tables.

Tenancy and security

- Row-level security is enabled on every table: `USING (tenant_id = current_setting('app.tenant_id')::uuid)`.
- The DB helper runs `SET LOCAL app.tenant_id` at the start of each transaction from the request context.
- Service roles get only `SELECT/INSERT/UPDATE` on their own schema; append-only tables (postings, audit) deny `UPDATE/DELETE`.

Performance and lifecycle

- High-volume tables (`postings`, `journal_entries`, `payment_events`, `card_authorizations`, `rider_locations`, `audit_logs`, `notifications`) are range-partitioned by month with `pg_partman`.
- Transactional outbox: each service has an `outbox_events` table written in the same transaction as its state change; Debezium publishes it to Kafka.

- Migrations: Prisma Migrate (TS services) and golang-migrate (ledger); expand-and-contract only, never destructive in one release.
- Read replicas serve admin lists and statements; the ledger primary serves posting and balance checks.

Shared tables in every service schema

Table	Columns
<code>outbox_events</code>	aggregate_type text, aggregate_id uuid, event_type text, payload jsonb, headers jsonb, published_at timestampz
<code>idempotency_keys</code>	key text, request_hash text, response_code int, response_body jsonb, locked_until timestampz, expires_at timestampz — unique (tenant_id, key)
<code>inbox_events</code>	event_id uuid, topic text, processed_at timestampz — unique (tenant_id, event_id) for idempotent consumers

4. Tables — Tenancy, Identity, Admin

These tables define who the tenant is, who its users are and who its staff are. The standard columns from section 3 are omitted.

Schema `tenancy` (tenant-service)

Table	Columns	Keys and indexes
<code>tenants</code> (no tenant_id)	code text, legal_name text, display_name text, country char(2), status text (ONBOARDING, ACTIVE, SUSPENDED, OFFBOARDED), deployment_model text (SHARED, DEDICATED), region text, current_profile_version int	unique(code)

Table	Columns	Keys and indexes
<code>tenant_profiles</code>	profile_version int, brand jsonb, market jsonb, compliance jsonb, products jsonb, fees_ref text, status text (DRAFT, ACTIVE, ARCHIVED), approved_by uuid, activated_at timestampz	unique(tenant_id, profile_version)
<code>tenant_adapters</code>	partner_type text (BANK, CARD_ISSUER, KYC, ACQUIRER, BILLER, SMS, EMAIL, PUSH), provider text, priority int, config jsonb, secret_ref text, status text (ACTIVE, DISABLED)	unique(tenant_id, partner_type, provider)
<code>tenant_domains</code>	domain text, kind text (API, ADMIN, MERCHANT, WEB), tls_status text	unique(domain)
<code>tenant_deployments</code>	environment text (STAGING, PROD), cluster text, region text, app_version text, helm_values_ref text, status text, deployed_at timestampz	index(tenant_id, environment)
<code>tenant_apps</code>	app_type text (CONSUMER, MERCHANT, RIDER), platform text (IOS, ANDROID), bundle_id text, store_account_ref text, min_supported_version text, latest_version text	unique(tenant_id, app_type, platform)
<code>tenant_usage_daily</code>	usage_date date, active_users int, txn_count int, txn_volume_minor bigint, currency char(3)	unique(tenant_id, usage_date, currency)

Schema `identity` (auth-service)

Table	Columns	Keys and indexes
<code>users</code>	user_type text (CUSTOMER, MERCHANT_STAFF, RIDER), phone_enc bytea, phone_hash text, email_enc bytea, email_hash text, first_name text, last_name text, dob_enc bytea, locale text, status text (PENDING, ACTIVE, FROZEN, BLOCKED, CLOSED), kyc_tier smallint, referral_code text, last_login_at timestamptz	unique(tenant_id, phone_hash); unique(tenant_id, referral_code)
<code>credentials</code>	user_id uuid, pin_hash text (Argon2id), pin_failed_count smallint, pin_locked_until timestamptz, biometric_enabled bool, password_hash text (merchant web only)	unique(user_id)
<code>devices</code>	user_id uuid, device_id text, platform text, model text, os_version text, app_version text, public_key text (device binding), push_token text, trusted bool, first_seen_at, last_seen_at, revoked_at timestamptz	unique(tenant_id, user_id, device_id)
<code>sessions</code>	user_id uuid, device_id text, refresh_token_hash text, ip inet, user_agent text, expires_at timestamptz, revoked_at timestamptz	index(user_id), unique(refresh_token_hash)

Table	Columns	Keys and indexes
<code>otp_challenges</code>	phone_hash text, purpose text (REGISTER, LOGIN, RESET_PIN, NEW_DEVICE, STEP_UP), code_hash text, channel text (SMS, WHATSAPP), attempts smallint, expires_at, verified_at timestamptz	index(tenant_id, phone_hash, purpose)
<code>login_events</code> (partitioned)	user_id uuid, device_id text, ip inet, geo_country char(2), result text (SUCCESS, FAILED, BLOCKED), reason text	index(tenant_id, user_id, created_at)
<code>user_addresses</code>	user_id uuid, label text, line1_enc bytea, line2_enc bytea, city text, country char(2), location geography(Point), is_default bool	index(user_id)
<code>referrals</code>	referrer_id uuid, referee_id uuid, status text (PENDING, QUALIFIED, REWARDED), reward_payment_id uuid	unique(referee_id)

Schemas `admin` and `audit` (admin-bff, audit-service)

Table	Columns	Keys and indexes
<code>admin_users</code>	scope text (PLATFORM, TENANT), tenant_id nullable for PLATFORM, email text, name text, keycloak_id text, status text (ACTIVE, DISABLED), last_login_at timestamptz	unique(scope, tenant_id, email)

Table	Columns	Keys and indexes
<code>permissions</code> (global)	code text (e.g. <code>kyc.case.approve</code>), module text, action text (VIEW, CREATE, EDIT, APPROVE, EXPORT, TOGGLE), description text	unique(code)
<code>roles</code>	name text, scope text, is_system bool	unique(tenant_id, name)
<code>role_permissions</code>	role_id uuid, permission_id uuid	pk(role_id, permission_id)
<code>admin_user_roles</code>	admin_user_id uuid, role_id uuid	pk(admin_user_id, role_id)
<code>approval_policies</code>	action_type text, min_approvers smallint, approver_role_id uuid, amount_threshold_minor bigint, currency char(3), enabled bool	unique(tenant_id, action_type)
<code>approval_requests</code>	action_type text, target_type text, target_id uuid, payload jsonb, before jsonb, after jsonb, reason text, maker_id uuid, status text (PENDING, APPROVED, REJECTED, EXPIRED, EXECUTED, FAILED), checker_id uuid, decided_at, expires_at timestamp, idempotency_key text	index(tenant_id, status, created_at)
<code>audit_logs</code> (append-only, partitioned)	actor_type text, actor_id uuid, action text, target_type text, target_id uuid, before jsonb, after jsonb, reason text, ip inet, user_agent text, request_id text	index(tenant_id, target_type, target_id), index(tenant_id, actor_id, created_at)
<code>pii_reveal_logs</code>	admin_user_id uuid, subject_type text, subject_id uuid, field text, reason text	index(tenant_id, subject_id)

Table	Columns	Keys and indexes
<code>api_keys</code>	owner_type text (MERCHANT, TENANT), owner_id uuid, key_prefix text, key_hash text, scopes text[], ip_allowlist inet[], last_used_at, revoked_at timestamptz	unique(key_prefix)

5. Tables — KYC, KYB, Risk and AML

KYC decides a user's tier, KYB decides whether a business can trade, and risk decides every transaction in real time. Velocity counters live in Redis; everything else is below.

Schema `kyc` (kyc-service)

Table	Columns	Keys and indexes
<code>kyc_profiles</code>	user_id uuid, current_tier smallint, full_name_enc bytea, dob_enc bytea, nationality char(2), id_number_hash text, address_enc bytea, occupation text, source_of_funds text, pep bool, risk_rating text (LOW, MEDIUM, HIGH), next_review_at timestamptz	unique(tenant_id, user_id); unique(tenant_id, id_number_hash)
<code>kyc_applications</code>	user_id uuid, target_tier smallint, status text (DRAFT, SUBMITTED, IN_PROGRESS, AUTO_APPROVED, PENDING_REVIEW, APPROVED, REJECTED, RESUBMIT_REQUESTED), provider text, provider_applicant_id text, submitted_at, decided_at timestamptz, decision_reason_code text	index(tenant_id, status, submitted_at); index(user_id)

Table	Columns	Keys and indexes
kyc_documents	application_id uuid, doc_type text (NATIONAL_ID, PASSPORT, DRIVING_LICENCE, RESIDENCE_PERMIT, PROOF_OF_ADDRESS, SELFIE, LIVENESS_VIDEO), side text (FRONT, BACK), file_id uuid, extracted jsonb, document_number_enc bytea, document_number_hash text, issuing_country char(2), expiry_date date, status text	index(application_id)
kyc_checks	application_id uuid, check_type text (OCR, AUTHENTICITY, LIVENESS, FACE_MATCH, SANCTIONS, PEP, ADVERSE_MEDIA, ADDRESS), provider text, result text (PASS, FAIL, REVIEW), score numeric(5,2), raw_file_id uuid, completed_at timestamptz	index(application_id)
kyc_cases	application_id uuid, flag_reasons text[], priority smallint, assigned_to uuid, sla_due_at timestamptz, status text (OPEN, IN_REVIEW, AWAITING_APPROVAL, CLOSED), decision text (APPROVE, REJECT, RESUBMIT), decision_reason_code text, notes text, approval_request_id uuid	index(tenant_id, status, sla_due_at); index(assigned_to)
kyc_reason_codes	code text, applies_to text (REJECT, RESUBMIT, FLAG), label_en text, label_ar text, active bool	unique(tenant_id, code)

Table	Columns	Keys and indexes
businesses	owner_user_id uuid, legal_name text, trade_name text, registration_number_hash text, registration_country char(2), legal_form text, mcc char(4), website text, status text (DRAFT, SUBMITTED, PENDING_REVIEW, APPROVED, REJECTED, SUSPENDED), risk_rating text, approved_at timestampz	unique(tenant_id, registration_number_hash)
business_documents	business_id uuid, doc_type text (REGISTRATION, TRADE_LICENCE, ARTICLES, TAX_CERTIFICATE, BANK_LETTER), file_id uuid, expiry_date date, status text	index(business_id)
business_ubos	business_id uuid, user_id uuid null, full_name_enc bytea, ownership_pct numeric(5,2), is_director bool, kyc_application_id uuid, screening_status text	index(business_id)

Schema **risk** (risk-service)

Table	Columns	Keys and indexes
risk_rules	code text, name text, stage text (ONBOARDING, LOGIN, PRE_TXN, POST_TXN), expression jsonb, action text (ALLOW, CHALLENGE, HOLD, BLOCK, ALERT), severity text, rule_version int, status text (DRAFT, ACTIVE, DISABLED), approved_by uuid	unique(tenant_id, code, rule_version)

Table	Columns	Keys and indexes
<code>risk_decisions</code> (partitioned)	subject_type text, subject_id uuid, stage text, score smallint, decision text, matched_rules text[], latency_ms int, context jsonb	index(tenant_id, subject_id, created_at)
<code>risk_scores</code>	entity_type text (USER, MERCHANT, DEVICE), entity_id text, score smallint, factors jsonb	unique(tenant_id, entity_type, entity_id)
<code>aml_alerts</code>	rule_code text, entity_type text, entity_id uuid, severity text, status text (NEW, TRIAGED, ESCALATED, CLOSED_FALSE_POSITIVE, CLOSED_TO_CASE), case_id uuid, details jsonb, assigned_to uuid	index(tenant_id, status, severity)
<code>aml_cases</code>	title text, entity_type text, entity_id uuid, assigned_to uuid, status text (OPEN, INVESTIGATING, PENDING_DECISION, CLOSED), decision text (NO_ACTION, FREEZE, EXIT, STR_FILED), str_reference text, closed_at timestampz	index(tenant_id, status)
<code>aml_case_notes</code>	case_id uuid, author_id uuid, body text, file_ids uuid[]	index(case_id)
<code>sanctions_lists</code> (global)	source text (UN, OFAC, EU, UK, LOCAL), list_version text, fetched_at timestampz, entries_count int	unique(source, list_version)
<code>sanctions_entries</code> (global)	list_id uuid, name_normalized text, aliases text[], dob date, nationality char(2), program text	GIN trigram index on name_normalized

Table	Columns	Keys and indexes
screening_results	subject_type text, subject_id uuid, entry_id uuid, match_score numeric(5,2), status text (POTENTIAL, CONFIRMED, FALSE_POSITIVE), reviewed_by uuid, reviewed_at timestampz	index(tenant_id, status)
blocklist_entries	entry_type text (PHONE_HASH, EMAIL_HASH, DEVICE_ID, IBAN_HASH, CARD_BIN, IP, MERCHANT_ID), value text, list text (BLOCK, ALLOW), reason text, expires_at timestampz	unique(tenant_id, entry_type, value)

6. Tables — Ledger, Wallet, Payments, Bills

The ledger schema is the only place balances exist. Wallets and payments point to ledger accounts and journal entries; they never store balances of their own.

Schema `ledger` (ledger-service, Go)

Table	Columns	Keys and indexes
<code>ledger_accounts</code>	code text, owner_type text (USER, MERCHANT, RIDER, AGENT, SYSTEM), owner_id uuid null, account_type text (CUSTOMER_WALLET, MERCHANT_WALLET, RIDER_WALLET, AGENT_FLOAT, SAFEGUARDING, BANK_CLEARING, CARD_SETTLEMENT, FEE_INCOME, COMMISSION_INCOME, INTERCHANGE_INCOME, FX_POSITION, REWARDS_EXPENSE, PAYOUT_PAYABLE, BILLER_PAYABLE, COD_PAYABLE, SUSPENSE), currency char(3), normal_balance text (DEBIT, CREDIT), allow_negative bool, status text	unique(tenant_id, owner_type, owner_id, account_type, currency); unique(tenant_id, code)
<code>journal_entries</code> (append-only, partitioned)	entry_type text (P2P, TOPUP, WITHDRAWAL, BILL_PAY, QR_PAY, ORDER_CAPTURE, CARD_CAPTURE, FEE, REFUND, REVERSAL, ADJUSTMENT, PAYOUT, FX, COD), reference_type text, reference_id uuid, description text, idempotency_key text, reverses_entry_id uuid, effective_at timestampz, posted_by text, approval_request_id uuid	unique(tenant_id, idempotency_key); index(tenant_id, reference_type, reference_id)

Table	Columns	Keys and indexes
<code>postings</code> (append-only, partitioned)	entry_id uuid, account_id uuid, direction text (DEBIT, CREDIT), amount_minor bigint CHECK > 0, currency char(3), balance_after_minor bigint	index(account_id, created_at); index(entry_id)
<code>ledger_balances</code>	account_id uuid, posted_minor bigint, held_minor bigint, available_minor bigint (generated), last_entry_id uuid	pk(account_id)
<code>holds</code>	account_id uuid, amount_minor bigint, currency char(3), reason_type text (CARD_AUTH, ORDER, WITHDRAWAL, PAYMENT), reference_id uuid, status text (ACTIVE, CAPTURED, RELEASED, EXPIRED), expires_at timestamptz, captured_entry_id uuid	index(account_id, status); index(tenant_id, status, expires_at)
<code>fx_rates</code>	base char(3), quote char(3), rate numeric(18,8), margin_bps int, source text, valid_from, valid_to timestamptz	index(tenant_id, base, quote, valid_from)
<code>recon_runs</code>	run_type text (INTERNAL, BANK, CARD, SAFEGUARDING), run_date date, status text, matched int, mismatches int, report_file_id uuid	unique(tenant_id, run_type, run_date)

The entry is balanced at write time: in one transaction the service locks the affected `ledger_balances` rows (ordered by account_id to avoid deadlocks), inserts the entry and postings, checks that debits equal credits per currency, updates balances and writes the outbox event.

Schema `wallet` (wallet-service)

Table	Columns	Keys and indexes
<code>wallets</code>	owner_type text, owner_id uuid, currency char(3), ledger_account_id uuid, status text (ACTIVE, FROZEN, CLOSED), is_default bool, frozen_reason text	unique(tenant_id, owner_type, owner_id, currency)
<code>limit_rules</code>	scope text (TIER, SEGMENT, USER, MERCHANT), scope_value text, service text (P2P, TOPUP, WITHDRAWAL, BILL, QR, CARD, CASH_OUT, ALL), period text (PER_TXN, DAILY, MONTHLY), max_amount_minor bigint, max_count int, currency char(3), source text (PROFILE, OVERRIDE), approval_request_id uuid, valid_to timestamptz	index(tenant_id, scope, scope_value, service)
<code>limit_usage</code>	wallet_id uuid, service text, period text, period_start date, amount_minor bigint, txn_count int	unique(wallet_id, service, period, period_start)
<code>beneficiaries</code>	user_id uuid, type text (WALLET, BANK), counterparty_user_id uuid, bank_account_id uuid, nickname text, last_used_at, deleted_at timestamptz	index(user_id)
<code>statements</code>	wallet_id uuid, period_start date, period_end date, file_id uuid, status text	unique(wallet_id, period_start)

Schema `payments` (payments-service)

Table	Columns	Keys and indexes
<code>payments</code>	type text (P2P, REQUEST, QR_MERCHANT, TOPUP_CARD, TOPUP_BANK, CASH_IN, CASH_OUT, WITHDRAWAL, BILL, ORDER, REFUND), status text (CREATED, RISK_CHECK, PENDING_USER, PENDING_REVIEW, AUTHORIZED, PROCESSING, COMPLETED, FAILED, DECLINED, REVERSED, PARTIALLY_REFUNDED, REFUNDED), payer_wallet_id uuid, payee_wallet_id uuid, amount_minor bigint, currency char(3), fee_minor bigint, fx_rate numeric(18,8), payee_amount_minor bigint, payee_currency char(3), rail text (INTERNAL, CARD, BANK, BILLER, AGENT), adapter text, external_ref text, reference_code text, note text, risk_decision_id uuid, hold_id uuid, journal_entry_id uuid, failure_code text, idempotency_key text, device_id text, completed_at timestamptz	unique(tenant_id, idempotency_key); unique(tenant_id, reference_code); index(payer_wallet_id, created_at); index(payee_wallet_id, created_at); index(tenant_id, status, created_at)
<code>payment_events</code> (partitioned)	payment_id uuid, from_status text, to_status text, event text, actor text, data jsonb	index(payment_id, created_at)

Table	Columns	Keys and indexes
payment_requests	requester_user_id uuid, payer_user_id uuid, amount_minor bigint, currency char(3), note text, status text (PENDING, PAID, DECLINED, EXPIRED, CANCELLED), payment_id uuid, expires_at timestampz	index(payer_user_id, status)
qr_codes	owner_type text (MERCHANT, USER), owner_id uuid, store_id uuid, kind text (STATIC, DYNAMIC), payload text (EMVCo), amount_minor bigint null, currency char(3), status text, expires_at timestampz	unique(tenant_id, payload)
fee_rules	service text, scope text (TIER, SEGMENT, MERCHANT, DEFAULT), scope_value text, fixed_minor bigint, percent_bps int, min_minor bigint, max_minor bigint, currency char(3), payer text (SENDER, RECEIVER, MERCHANT), valid_from, valid_to timestampz	index(tenant_id, service, scope)
funding_cards	user_id uuid, acquirer text, token text, brand text, last4 char(4), expiry_month smallint, expiry_year smallint, status text	index(user_id)
cash_agents	business_id uuid, name text, location geography(Point), float_wallet_id uuid, opening_hours jsonb, status text	GIST(location)

Table	Columns	Keys and indexes
payment_disputes	payment_id uuid, raised_by uuid, reason_code text, description text, status text (OPEN, INVESTIGATING, RESOLVED_REFUND, RESOLVED_NO_ACTION), resolution_payment_id uuid	index(tenant_id, status)

Schema bills (bills-service)

Table	Columns	Keys and indexes
billers	category text (TELECOM, ELECTRICITY, WATER, INTERNET, GOVERNMENT, EDUCATION, OTHER), name_en text, name_ar text, logo_file_id uuid, adapter text, provider_biller_code text, inquiry_supported bool, sort_order int, status text	unique(tenant_id, adapter, provider_biller_code)
biller_fields	biller_id uuid, field_key text, label_en text, label_ar text, input_type text, regex text, required bool, sort_order int	index(biller_id)
bill_payments	payment_id uuid, biller_id uuid, account_ref text, inquiry_ref text, due_amount_minor bigint, paid_amount_minor bigint, currency char(3), status text, provider_ref text, receipt jsonb	unique(payment_id)
saved_bills	user_id uuid, biller_id uuid, account_ref text, nickname text, remind_day smallint	unique(user_id, biller_id, account_ref)

7. Tables — Banking and Cards

These tables record every movement across external rails and match it back to the ledger. Only processor tokens and last four digits of cards are stored, never full card numbers.

Schema `banking` (banking-service)

Table	Columns	Keys and indexes
<code>bank_partners</code>	adapter text, bank_name text, swift_bic text, country char(2), capabilities text[] (PAYOUT, VIRTUAL_ACCOUNT, NAME_ENQUIRY, STATEMENT_API, INSTANT), cutoff_times jsonb, holiday_calendar text, status text	unique(tenant_id, adapter)
<code>internal_bank_accounts</code>	partner_id uuid, purpose text (SAFEGUARDING, SETTLEMENT, FEES), iban_enc bytea, account_number_enc bytea, currency char(3), ledger_account_id uuid, status text	unique(tenant_id, partner_id, purpose, currency)

Table	Columns	Keys and indexes
linked_bank_accounts	owner_type text (USER, MERCHANT, RIDER), owner_id uuid, bank_name text, iban_enc bytea, iban_hash text, account_number_enc bytea, holder_name text, currency char(3), verification_status text (PENDING, VERIFIED, FAILED), verification_method text (NAME_ENQUIRY, PENNY_TEST, DOCUMENT), is_default bool, cooling_until timestampz, deleted_at timestampz	unique(tenant_id, owner_id, iban_hash)
virtual_accounts	owner_type text, owner_id uuid, partner_id uuid, iban_enc bytea, iban_hash text, reference text, currency char(3), status text	unique(tenant_id, iban_hash); unique(tenant_id, reference)
bank_transfers	direction text (INBOUND, OUTBOUND), payment_id uuid, partner_id uuid, linked_account_id uuid, virtual_account_id uuid, amount_minor bigint, currency char(3), status text (CREATED, SUBMITTED, ACCEPTED, SETTLED, REJECTED, RETURNED, UNMATCHED), rail text (INSTANT, ACH, SWIFT, BOOK), rail_ref text, batch_id uuid, value_date date, failure_code text	unique(tenant_id, partner_id, rail_ref); index(tenant_id, status, created_at)

Table	Columns	Keys and indexes
bank_file_batches	partner_id uuid, direction text (OUT, IN), file_type text (PAIN001, PAIN002, CAMT053, MT940), file_id uuid, records int, status text (CREATED, SENT, ACKED, FAILED, PROCESSED)	index(tenant_id, partner_id, created_at)
bank_statements	partner_id uuid, internal_account_id uuid, statement_date date, opening_minor bigint, closing_minor bigint, currency char(3), file_id uuid, status text	unique(internal_account_id, statement_date)
statement_lines	statement_id uuid, value_date date, amount_minor bigint, direction text (CREDIT, DEBIT), reference text, counterparty_name text, counterparty_iban_hash text, match_status text (UNMATCHED, MATCHED, SUSPENSE, IGNORED), matched_transfer_id uuid	index(statement_id); index(tenant_id, match_status)

Table	Columns	Keys and indexes
<code>recon_exceptions</code>	source text (BANK, CARD, INTERNAL), reference_type text, reference_id uuid, exception_type text (UNMATCHED_CREDIT, AMOUNT_MISMATCH, MISSING_IN_BANK, MISSING_IN_LEDGER, DUPLICATE), amount_minor bigint, currency char(3), status text (OPEN, RESOLVED), resolved_by uuid, resolution text	index(tenant_id, status, source)

Schema `cards` (cards-service)

Table	Columns	Keys and indexes
<code>card_programs</code>	issuer_adapter text, name text, bin text, network text (VISA, MASTERCARD), form text (VIRTUAL, PHYSICAL), product text (PREPAID, DEBIT), currency char(3), design_ref text, min_kyc_tier smallint, fees jsonb, status text	unique(tenant_id, issuer_adapter, bin, form)

Table	Columns	Keys and indexes
cards	user_id uuid, wallet_id uuid, program_id uuid, processor_card_token text, last4 char(4), expiry_month smallint, expiry_year smallint, cardholder_name text, status text (REQUESTED, ISSUED, SHIPPED, ACTIVE, FROZEN, BLOCKED, EXPIRED, REPLACED, CLOSED), block_reason text (LOST, STOLEN, FRAUD, DAMAGED, ADMIN), replaced_by_card_id uuid, activated_at timestamptz	unique(tenant_id, processor_card_token); index(user_id)
card_controls	card_id uuid, ecommerce bool, atm bool, contactless bool, international bool, mcc_blocklist text[], country_allowlist text[], per_txn_limit_minor bigint, daily_limit_minor bigint, monthly_limit_minor bigint	pk(card_id)
card_shipments	card_id uuid, address_id uuid, carrier text, tracking_number text, status text (PENDING, PRINTED, SHIPPED, DELIVERED, RETURNED), shipped_at, delivered_at timestamptz	index(card_id)

Table	Columns	Keys and indexes
card_authorizations (partitioned)	card_id uuid, processor_auth_id text, auth_type text (AUTH, INCREMENTAL, REVERSAL, PARTIAL_REVERSAL, ADVICE, FORCE_POST), amount_minor bigint, currency char(3), billing_amount_minor bigint, billing_currency char(3), mcc char(4), merchant_name text, merchant_city text, merchant_country char(2), entry_mode text, is_ecommerce bool, three_ds bool, decision text (APPROVED, DECLINED), decline_code text, auth_code text, stan text, rrn text, hold_id uuid, risk_decision_id uuid, latency_ms int	unique(tenant_id, processor_auth_id, auth_type); index(card_id, created_at)
card_transactions	card_id uuid, authorization_id uuid, clearing_ref text, cleared_amount_minor bigint, currency char(3), fx_rate numeric(18,8), fee_minor bigint, journal_entry_id uuid, status text (PENDING, CLEARED, REVERSED, REFUNDED), cleared_at timestampz	unique(tenant_id, clearing_ref); index(card_id, created_at)

Table	Columns	Keys and indexes
<code>card_disputes</code>	card_transaction_id uuid, reason_code text, amount_minor bigint, status text (DRAFT, SUBMITTED, PROVISIONAL_CREDIT, WON, LOST, WITHDRAWN), processor_dispute_id text, provisional_payment_id uuid, deadline_at timestamptz, evidence_file_ids uuid[]	index(tenant_id, status, deadline_at)
<code>card_tokens</code>	card_id uuid, wallet_provider text (APPLE_PAY, GOOGLE_PAY, SAMSUNG_PAY), token_ref text, device_name text, status text (ACTIVE, SUSPENDED, DELETED)	unique(tenant_id, token_ref)
<code>card_settlements</code>	settlement_date date, network text, currency char(3), gross_minor bigint, interchange_minor bigint, scheme_fees_minor bigint, net_minor bigint, file_id uuid, journal_entry_id uuid	unique(tenant_id, settlement_date, network, currency)

8. Tables — Marketplace, Delivery, Payouts, Platform, Notifications, Support, Files, Reporting

These are the commerce and operations tables. Money still settles only through ledger entries, which these tables reference by `journal_entry_id` or `payment_id`.

Schema marketplace (marketplace-service)

Table	Columns	Keys and indexes
merchants	business_id uuid, owner_user_id uuid, display_name text, category_id uuid, commission_bps int, settlement_wallet_id uuid, payout_schedule text (DAILY, WEEKLY, ON_DEMAND), reserve_bps int, rating numeric(3,2), status text (PENDING, ACTIVE, SUSPENDED, CLOSED)	unique(tenant_id, business_id)
merchant_staff	merchant_id uuid, user_id uuid, role text (OWNER, MANAGER, CASHIER), store_ids uuid[]	unique(merchant_id, user_id)
stores	merchant_id uuid, name_en text, name_ar text, location geography(Point), address text, phone text, opening_hours jsonb, delivery_radius_m int, min_order_minor bigint, prep_time_min smallint, is_open bool, status text	GIST(location); index(merchant_id)
categories	parent_id uuid, name_en text, name_ar text, icon_file_id uuid, sort_order int, status text	index(tenant_id, parent_id)

Table	Columns	Keys and indexes
products	merchant_id uuid, store_id uuid, category_id uuid, sku text, name_en text, name_ar text, description_en text, description_ar text, price_minor bigint, compare_at_minor bigint, currency char(3), track_stock bool, stock_qty int, image_file_ids uuid[], status text (DRAFT, PENDING_REVIEW, ACTIVE, REJECTED, ARCHIVED), deleted_at timestampz	unique(store_id, sku); index(tenant_id, category_id, status)
product_options	product_id uuid, group_name text, name text, price_delta_minor bigint, required bool, sort_order int	index(product_id)
carts	user_id uuid, store_id uuid, items jsonb, expires_at timestampz	unique(user_id, store_id)

Table	Columns	Keys and indexes
orders	order_number text, user_id uuid, store_id uuid, merchant_id uuid, status text (PLACED, ACCEPTED, PREPARING, READY, PICKED_UP, DELIVERED, CANCELLED, REJECTED), fulfilment text (DELIVERY, PICKUP), subtotal_minor, delivery_fee_minor, service_fee_minor, discount_minor, total_minor bigint, currency char(3), payment_method text (WALLET, CARD, COD), payment_id uuid, hold_id uuid, delivery_address_id uuid, promotion_id uuid, notes text, cancel_reason text, placed_at, delivered_at timestampz	unique(tenant_id, order_number); index(user_id, created_at); index(store_id, status)
order_items	order_id uuid, product_id uuid, name_snapshot text, quantity int, unit_price_minor bigint, options jsonb, total_minor bigint	index(order_id)
order_events	order_id uuid, from_status text, to_status text, actor_type text, actor_id uuid	index(order_id, created_at)

Table	Columns	Keys and indexes
promotions	code text, promo_type text (PERCENT, FIXED, FREE_DELIVERY, CASHBACK), value int, max_discount_minor bigint, min_order_minor bigint, funded_by text (PLATFORM, MERCHANT), merchant_id uuid, budget_minor bigint, used_minor bigint, per_user_limit int, target jsonb, starts_at, ends_at timestampz, status text	unique(tenant_id, code)
promotion_redemptions	promotion_id uuid, user_id uuid, order_id uuid, payment_id uuid, discount_minor bigint	index(promotion_id, user_id)
commissions	order_id uuid, merchant_id uuid, gross_minor bigint, commission_minor bigint, rider_share_minor bigint, platform_delivery_minor bigint, journal_entry_id uuid	unique(order_id)
reviews	order_id uuid, user_id uuid, target_type text (STORE, RIDER), target_id uuid, rating smallint, comment text, status text (VISIBLE, HIDDEN)	unique(order_id, target_type)

Schemas `delivery` and `payouts`

Table	Columns	Keys and indexes
<code>riders</code>	<code>user_id</code> uuid, <code>vehicle_type</code> text (BICYCLE, MOTORBIKE, CAR), <code>vehicle_plate</code> text, <code>licence_expiry</code> date, <code>zone_id</code> uuid, <code>wallet_id</code> uuid, <code>rating</code> numeric(3,2), <code>availability</code> text (OFFLINE, ONLINE, BUSY), <code>status</code> text (ONBOARDING, ACTIVE, SUSPENDED, OFFBOARDED)	<code>unique(tenant_id, user_id)</code>
<code>rider_documents</code>	<code>rider_id</code> uuid, <code>doc_type</code> text, <code>file_id</code> uuid, <code>expiry_date</code> date, <code>status</code> text	<code>index(rider_id)</code>
<code>zones</code>	<code>name</code> text, <code>area</code> geography(Polygon), <code>delivery_fee_rules</code> jsonb, <code>status</code> text	<code>GIST(area)</code>
<code>deliveries</code>	<code>order_id</code> uuid, <code>rider_id</code> uuid, <code>status</code> text (PENDING_ASSIGNMENT, OFFERED, ASSIGNED, AT_PICKUP, PICKED_UP, AT_DROPOFF, DELIVERED, FAILED, CANCELLED), <code>pickup_location</code> , <code>dropoff_location</code> geography(Point), <code>distance_m</code> int, <code>eta_at</code> , <code>assigned_at</code> , <code>picked_up_at</code> , <code>delivered_at</code> timestamptz, <code>proof_file_id</code> uuid, <code>cod_amount_minor</code> bigint	<code>unique(order_id);</code> <code>index(rider_id, status)</code>
<code>delivery_offers</code>	<code>delivery_id</code> uuid, <code>rider_id</code> uuid, <code>status</code> text (SENT, ACCEPTED, REJECTED, EXPIRED), <code>expires_at</code> timestamptz	<code>index(delivery_id);</code> <code>index(rider_id, status)</code>

Table	Columns	Keys and indexes
rider_locations (partitioned, 90-day retention)	rider_id uuid, location geography(Point), heading smallint, speed_kmh smallint, accuracy_m smallint, recorded_at timestamptz	index(rider_id, recorded_at)
rider_earnings	rider_id uuid, delivery_id uuid, earning_type text (DELIVERY_FEE, TIP, BONUS, PENALTY), amount_minor bigint, currency char(3), journal_entry_id uuid	index(rider_id, created_at)
cod_collections	rider_id uuid, delivery_id uuid, amount_minor bigint, currency char(3), status text (COLLECTED, DEPOSITED, NETTED), deposit_ref text	index(rider_id, status)
payout_batches	owner_type text (MERCHANT, RIDER), run_at timestamptz, currency char(3), payout_count int, total_minor bigint, status text	index(tenant_id, run_at)
payouts	batch_id uuid, owner_type text, owner_id uuid, wallet_id uuid, linked_bank_account_id uuid, amount_minor bigint, reserve_minor bigint, currency char(3), status text (PENDING, SENT, PAID, FAILED, RETURNED), bank_transfer_id uuid, statement_file_id uuid	index(tenant_id, owner_id, created_at)

Schemas `platform`, `notify`, `support`, `files`, `reporting`

Table	Columns	Keys and indexes
<code>feature_flags</code>	flag_key text, description text, enabled bool, kill_switch bool, disabled_message jsonb (per locale), updated_by uuid	unique(tenant_id, flag_key)
<code>flag_targets</code>	flag_id uuid, rule_order smallint, conditions jsonb (country, platform, app version range, KYC tier, segment, user list), rollout_pct smallint, starts_at, ends_at timestampz, enabled bool	index(flag_id, rule_order)
<code>segments</code>	name text, definition jsonb, estimated_size int, refreshed_at timestampz	unique(tenant_id, name)
<code>sdui_screens</code>	screen_key text (home, services, marketplace_home, wallet), description text	unique(tenant_id, screen_key)
<code>sdui_versions</code>	screen_id uuid, version int, schema_version int, content jsonb, segment_id uuid, status text (DRAFT, PENDING_APPROVAL, SCHEDULED, PUBLISHED, ARCHIVED), publish_at timestampz, published_by uuid, approval_request_id uuid	unique(screen_id, version)
<code>app_versions</code>	app_type text, platform text, version text, build int, force_update bool, soft_update bool, release_notes jsonb	unique(tenant_id, app_type, platform, version)

Table	Columns	Keys and indexes
<code>maintenance_windows</code>	scope text (ALL, SERVICE), service_key text, message jsonb, starts_at, ends_at timestampz	index(tenant_id, starts_at)
<code>translations</code>	namespace text, key text, locale text, value text	unique(tenant_id, namespace, key, locale)
<code>notification_templates</code>	event_type text, channel text (PUSH, SMS, EMAIL, IN_APP), locale text, title text, body text, variables text[], template_version int, active bool	unique(tenant_id, event_type, channel, locale, template_version)
<code>notifications</code> (partitioned)	user_id uuid, channel text, template_id uuid, event_id uuid, title text, body text, data jsonb, status text (QUEUED, SENT, DELIVERED, FAILED, READ), provider text, provider_message_id text, sent_at, read_at timestampz	index(user_id, created_at); unique(tenant_id, event_id, channel)
<code>notification_preferences</code>	user_id uuid, category text (TRANSACTIONS, SECURITY, MARKETING, ORDERS), channel text, enabled bool	unique(user_id, category, channel)
<code>campaigns</code>	name text, segment_id uuid, channel text, template_id uuid, schedule_at timestampz, status text (DRAFT, PENDING_APPROVAL, SCHEDULED, SENDING, SENT, CANCELLED), sent_count int, approval_request_id uuid	index(tenant_id, status)

Table	Columns	Keys and indexes
tickets	ticket_number text, requester_type text, requester_id uuid, category text, subject text, status text (OPEN, PENDING, ON_HOLD, RESOLVED, CLOSED), priority text, assigned_to uuid, linked_payment_id uuid, linked_order_id uuid, sla_due_at timestampz	unique(tenant_id, ticket_number); index(tenant_id, status, priority)
ticket_messages	ticket_id uuid, author_type text (CUSTOMER, AGENT, SYSTEM), author_id uuid, body text, file_ids uuid[], internal bool	index(ticket_id, created_at)
support_macros	name text, body_en text, body_ar text, category text	unique(tenant_id, name)
files	owner_type text, owner_id uuid, purpose text (KYC, KYB, PRODUCT_IMAGE, DELIVERY_PROOF, EXPORT, STATEMENT, EVIDENCE, BRAND), bucket text, object_key text, mime text, size_bytes bigint, sha256 text, scan_status text (PENDING, CLEAN, INFECTED), retention_until date	unique(bucket, object_key); index(tenant_id, owner_id)
report_definitions	code text, name text, format text (CSV, XLSX, PDF), query_ref text, params_schema jsonb, is_regulatory bool	unique(tenant_id, code)

Table	Columns	Keys and indexes
<code>report_schedules</code>	definition_id uuid, cron text, params jsonb, recipients jsonb, active bool	index(definition_id)
<code>report_runs</code>	definition_id uuid, params jsonb, status text (QUEUED, RUNNING, DONE, FAILED), file_id uuid, requested_by uuid, schedule_id uuid, completed_at timestamptz	index(tenant_id, definition_id, created_at)

9. API Conventions

All external APIs are REST/JSON over HTTPS through Kong. The gateway resolves the tenant, validates the token, applies rate limits and forwards to the owning service. Every endpoint in sections 10–12 follows these rules.

Base path	Consumers	Auth
<code>/mobile/v1</code>	Consumer app	User JWT (EdDSA, 10 min) + device binding
<code>/mobile/v1/merchant</code>	Merchant app	Merchant staff JWT
<code>/mobile/v1/rider</code>	Rider app	Rider JWT
<code>/merchant/v1</code>	Merchant servers (integrations)	API key + HMAC-SHA256 request signature
<code>/admin/v1</code>	Tenant admin portal (via admin-bff)	Keycloak OIDC session + TOTP, RBAC permission per endpoint
<code>/console/v1</code>	Platform console (via console-bff)	Keycloak OIDC (platform realm) + TOTP
<code>/webhooks/{partnerType}/{provider}</code>	Banks, processors, KYC, billers	Partner signature or mTLS

Headers

Header	Required on	Purpose
<code>Authorization: Bearer <jwt></code>	All authenticated calls	Identity; the JWT carries <code>tid</code> (tenant), <code>sub</code> , <code>dev</code> , <code>typ</code>
<code>X-Device-Id</code>	Mobile	Must match the device bound to the token
<code>X-App-Version</code> , <code>X-Platform</code>	Mobile	Force-update checks, flag targeting
<code>Accept-Language</code>	All	<code>ar</code> or <code>en</code> ; localizes errors and content
<code>Idempotency-Key</code>	Every POST that moves money or creates a resource	UUID from the client; replay returns the stored response for 24 h
<code>X-Step-Up-Token</code>	Endpoints marked Step-up	Short-lived token from PIN/biometric verification, bound to the request hash
<code>X-Request-Id</code>	Optional	Echoed back; used as trace correlation
<code>X-Signature</code> , <code>X-Timestamp</code> , <code>X-Key-Id</code>	Merchant API	HMAC over <code>method + path + timestamp + sha256(body)</code> ; 5-minute skew

Responses

- Success: resource or `{ "data": [...], "nextCursor": "..."}` for lists; cursor pagination with `?limit=` (max 100) and `?cursor=`.
- Money in APIs: `{ "amount": "10.00", "currency": "USD" }` (decimal string).
- Time: ISO 8601 UTC.
- Errors: `{ "error": { "code", "message", "details", "requestId" } }` with stable codes: `VALIDATION_FAILED`, `UNAUTHENTICATED`, `FORBIDDEN`, `NOT_FOUND`, `CONFLICT`, `DUPLICATE_REQUEST`, `INSUFFICIENT_FUNDS`, `LIMIT_EXCEEDED`,

KYC_REQUIRED, STEP_UP_REQUIRED, RISK_DECLINED, RISK_REVIEW, ACCOUNT_FROZEN, SERVICE_DISABLED, UPDATE_REQUIRED, MAINTENANCE, RATE_LIMITED, PARTNER_UNAVAILABLE.

- Maker-checker endpoints return 202 Accepted with { "approvalRequestId" } instead of applying the change.

Auth legend used in the endpoint tables

Code	Meaning
Public	No token (tenant still resolved from domain)
Reg	Registration/OTP token only
User	Consumer JWT
Step-up	User JWT + X-Step-Up-Token
Staff	Merchant staff JWT (role checked)
Rider	Rider JWT
Key	Merchant API key + signature
Admin:perm	Tenant staff with the named permission
MC	Maker-checker: returns 202 and needs approval

10. Endpoints — Mobile Consumer API (/mobile/v1)

The consumer app needs 116 endpoints, grouped below by owning service. Paths are relative to /mobile/v1.

App config and files (platform-service, file-service)

Method	Path	Auth	Purpose
GET	/app/config	Public	Tenant brand, enabled services, flags, min/latest version, maintenance status
GET	/app/screens/{screenKey}	User	SDUI screen JSON for the user's segment (ETag)

Method	Path	Auth	Purpose
GET	/app/translations?ns=	Public	Server-managed strings per locale
POST	/files/upload-url	User	Pre-signed upload URL + fileId for a given purpose

Auth and devices (auth-service)

Method	Path	Auth	Purpose
POST	/auth/otp/request	Public	Send OTP for REGISTER, LOGIN, NEW_DEVICE or RESET_PIN
POST	/auth/otp/verify	Public	Verify OTP; returns a registration token or login continuation
POST	/auth/register	Reg	Create user with name, PIN and device public key; returns tokens
POST	/auth/login	Public	Phone + PIN + device signature; returns tokens or requires OTP for new device
POST	/auth/token/refresh	Public	Rotate access and refresh tokens
POST	/auth/logout	User	Revoke the current session
POST	/auth/step-up	User	Verify PIN or biometric signature; returns a step-up token for one action
POST	/auth/pin/change	Step-up	Change PIN
POST	/auth/pin/reset	Reg	Reset PIN after OTP (with new-device cooling rules)
GET	/devices	User	List devices
DELETE	/devices/{deviceId}	Step-up	Revoke a device and its sessions

Profile (auth-service)

Method	Path	Auth	Purpose
GET	/me	User	Profile, KYC tier, status, locale
PATCH	/me	User	Update editable profile fields
GET	/me/addresses	User	List addresses
POST	/me/addresses	User	Add address
PATCH	/me/addresses/{id}	User	Edit address
DELETE	/me/addresses/{id}	User	Remove address
GET	/me/referrals	User	Referral code and status of referees
POST	/me/close	Step-up	Request account closure (balance must be zero)

KYC (kyc-service)

Method	Path	Auth	Purpose
GET	/kyc/status	User	Current tier, open application, required actions
GET	/kyc/requirements?tier=	User	Documents and checks needed for a tier (from tenant profile)
POST	/kyc/applications	User	Start an application for a target tier
POST	/kyc/applications/{id}/documents	User	Attach an uploaded file as a document (type, side)
POST	/kyc/applications/{id}/liveness-session	User	Provider SDK token for liveness and selfie
POST	/kyc/applications/{id}/submit	User	Submit for checks

Wallet and beneficiaries (wallet-service)

Method	Path	Auth	Purpose
GET	/wallets	User	Wallets with posted and available balances
POST	/wallets	User	Open a wallet in another enabled currency
GET	/wallets/{id}/transactions	User	History with filters (type, date, status)
GET	/transactions/{id}	User	Transaction detail
GET	/limits	User	Limits and remaining usage per service
GET	/statements	User	Generated statements
POST	/statements	User	Generate a statement for a period (PDF)
GET	/beneficiaries	User	Saved recipients
POST	/beneficiaries	User	Save a wallet or bank recipient
DELETE	/beneficiaries/{id}	User	Remove recipient
POST	/contacts/match	User	Hashed phone numbers in, registered wallets out (for P2P picker)

Payments, top-up, cash (payments-service)

Method	Path	Auth	Purpose
POST	/payments/quote	User	Fee, FX rate and total before confirming
POST	/payments/p2p	Step-up	Send money to a wallet
POST	/payments/requests	User	Request money
GET	/payments/requests?role=payer or ?role=requester	User	List requests I must pay or I sent

Method	Path	Auth	Purpose
POST	/payments/requests/{id}/pay	Step-up	Pay a request
POST	/payments/requests/{id}/decline	User	Decline a request
DELETE	/payments/requests/{id}	User	Cancel own request
GET	/qr/me	User	User's static receive QR
POST	/payments/qr/parse	User	Decode a QR into payee, amount and currency
POST	/payments/qr/pay	Step-up	Pay a merchant or user QR
GET	/payments/{id}	User	Payment status
GET	/payments/{id}/receipt	User	Receipt (JSON, PDF link)
POST	/payments/{id}/disputes	User	Raise a dispute on a wallet payment
GET	/funding-cards	User	Saved top-up cards
POST	/funding-cards/session	User	Acquirer tokenization session (card never touches backend)
DELETE	/funding-cards/{id}	User	Remove saved card
POST	/topups/card	Step-up	Top up by card; returns 3DS action if needed
GET	/topups/bank-details	User	Virtual IBAN or reference for bank top-up
GET	/cash-agents?lat=&lng=	User	Nearby cash-in/out agents
POST	/cash-out/codes	Step-up	One-time code for cash-out at an agent

Bank accounts and withdrawals (banking-service)

Method	Path	Auth	Purpose
GET	/bank-accounts	User	Linked accounts
POST	/bank-accounts	Step-up	Link an account (name enquiry runs automatically)
DELETE	/bank-accounts/{id}	Step-up	Unlink
POST	/withdrawals	Step-up	Withdraw to a verified linked account
GET	/withdrawals/{id}	User	Withdrawal status

Bills (bills-service)

Method	Path	Auth	Purpose
GET	/billers/categories	User	Categories
GET	/billers?category=	User	Billers
GET	/billers/{id}	User	Biller with input fields
POST	/bills/inquiry	User	Fetch due amount
POST	/bills/pay	Step-up	Pay a bill
GET	/saved-bills	User	Saved bills
POST	/saved-bills	User	Save a bill
DELETE	/saved-bills/{id}	User	Remove

Cards (cards-service)

Method	Path	Auth	Purpose
GET	/card-programs	User	Card products available to this user

Method	Path	Auth	Purpose
POST	/cards	Step-up	Request a virtual or physical card
GET	/cards	User	Cards
GET	/cards/{id}	User	Card detail (last4, status, controls)
POST	/cards/{id}/activate	Step-up	Activate a physical card
POST	/cards/{id}/freeze	User	Freeze
POST	/cards/{id}/unfreeze	Step-up	Unfreeze
POST	/cards/{id}/block	Step-up	Block as lost, stolen or damaged
POST	/cards/{id}/replace	Step-up	Replace a blocked or expired card
GET	/cards/{id}/controls	User	Controls
PATCH	/cards/{id}/controls	Step-up	Update e-com, ATM, contactless, international, limits
POST	/cards/{id}/reveal-session	Step-up	Token for processor secure display of PAN/CVV
POST	/cards/{id}/pin-session	Step-up	Token for processor PIN set/change SDK
POST	/cards/{id}/provisioning	Step-up	Apple Pay / Google Pay push-provisioning data
GET	/cards/{id}/transactions	User	Card transactions
POST	/cards/{id}/disputes	User	Raise a card dispute
GET	/cards/{id}/disputes	User	Dispute status

Marketplace (marketplace-service, delivery-service)

Method	Path	Auth	Purpose
GET	/marketplace/home	User	SDUI marketplace home
GET	/categories	User	Categories tree
GET	/stores? lat=&lng=&category=&q=	User	Nearby stores
GET	/stores/{id}	User	Store detail
GET	/stores/{id}/products	User	Store catalog
GET	/products/{id}	User	Product with options
GET	/search?q=	User	Search stores and products
GET	/carts/{storeId}	User	Cart
PUT	/carts/{storeId}	User	Replace cart items
DELETE	/carts/{storeId}	User	Clear cart
POST	/promotions/validate	User	Validate a promo code against the cart
POST	/orders	Step-up	Checkout: create order, hold funds (or COD)
GET	/orders	User	Orders
GET	/orders/{id}	User	Order detail
POST	/orders/{id}/cancel	User	Cancel before acceptance
GET	/orders/{id}/tracking	User	Rider position and ETA snapshot (live updates by socket)
POST	/orders/{id}/reviews	User	Rate store and rider

Notifications and support (notification-service, support-service)

Method	Path	Auth	Purpose
POST	/notifications/token	User	Register FCM token for the device
GET	/notifications	User	In-app inbox
POST	/notifications/read	User	Mark as read (ids or all)
GET	/notification-preferences	User	Preferences
PUT	/notification-preferences	User	Update preferences
GET	/support/tickets	User	Tickets
POST	/support/tickets	User	Open a ticket (optionally linked to payment or order)
GET	/support/tickets/{id}	User	Ticket with messages
POST	/support/tickets/{id}/messages	User	Reply

11. Endpoints — Merchant and Rider APIs

Merchants use two surfaces: the merchant app for staff, and a signed server API for integrated merchants. Riders have their own app API; live location goes over the socket (section 14), with a REST fallback.

Merchant app (/mobile/v1/merchant)

Method	Path	Auth	Purpose
GET	/onboarding	Staff	KYB progress and required items
POST	/onboarding/business	Staff	Submit business details and UBOs
POST	/onboarding/documents	Staff	Attach uploaded KYB documents
POST	/onboarding/submit	Staff	Submit KYB for review

Method	Path	Auth	Purpose
GET	/me	Staff	Merchant profile, role, stores
GET	/dashboard? storeId=&period=	Staff	Sales, orders, average ticket
GET	/balance	Staff (Owner, Manager)	Settlement wallet balance
GET	/transactions	Staff	Payments received, refunds
GET	/qr/static?storeId=	Staff	Store static QR
POST	/qr/dynamic	Staff	Dynamic QR for an amount
POST	/refunds	Step-up (Owner, Manager)	Full or partial refund
GET	/stores	Staff	Stores
PATCH	/stores/{id}	Staff (Manager)	Hours, open/closed, prep time
GET	/products?storeId=	Staff	Catalog
POST	/products	Staff (Manager)	Create product (goes to moderation if required)
PATCH	/products/{id}	Staff (Manager)	Edit product
PATCH	/products/{id}/stock	Staff	Update stock or availability
DELETE	/products/{id}	Staff (Manager)	Archive product
GET	/orders?status=	Staff	Incoming and active orders
POST	/orders/{id}/accept	Staff	Accept with prep time
POST	/orders/{id}/reject	Staff	Reject with reason (releases hold)
POST	/orders/{id}/ready	Staff	Mark ready for pickup (triggers dispatch)
GET	/staff	Staff (Owner)	Staff list

Method	Path	Auth	Purpose
POST	/staff	Step-up (Owner)	Invite staff with role and stores
DELETE	/staff/{id}	Step-up (Owner)	Remove staff
GET	/payouts	Staff (Owner)	Payout history
POST	/payouts/instant	Step-up (Owner)	On-demand payout (if enabled)
GET	/reports/settlement?date=	Staff (Owner, Manager)	Daily settlement report

Merchant server API (/merchant/v1)

Method	Path	Auth	Purpose
POST	/payment-intents	Key	Create a payment for checkout or dynamic QR
GET	/payment-intents/{id}	Key	Status
POST	/payment-intents/{id}/cancel	Key	Cancel an unpaid intent
POST	/refunds	Key	Refund a payment
GET	/refunds/{id}	Key	Refund status
GET	/orders	Key	Orders
GET	/orders/{id}	Key	Order
PATCH	/orders/{id}/status	Key	Accept, reject, ready
GET	/products	Key	Catalog
POST	/products	Key	Create product
PATCH	/products/{id}	Key	Update product
PATCH	/products/{id}/stock	Key	Stock sync
DELETE	/products/{id}	Key	Archive

Method	Path	Auth	Purpose
GET	/stores	Key	Stores
PATCH	/stores/{id}	Key	Store status and hours
GET	/balance	Key	Balance
GET	/transactions	Key	Transactions
GET	/payouts	Key	Payouts
GET	/reports/settlement?date=	Key	Settlement report (CSV)
GET	/webhook-endpoints	Key	Webhook endpoints
POST	/webhook-endpoints	Key	Register an endpoint and event types
DELETE	/webhook-endpoints/{id}	Key	Remove endpoint
POST	/webhook-endpoints/{id}/test	Key	Send a test event
GET	/events?since=	Key	Replay missed events

Outbound merchant webhook events: `payment_intent.succeeded`, `payment_intent.failed`, `refund.succeeded`, `refund.failed`, `order.placed`, `order.cancelled`, `payout.paid`, `payout.failed`. Each is signed with HMAC-SHA256, retried with backoff for 72 hours, and deduplicated by `eventId`.

Rider app (/mobile/v1/rider)

Method	Path	Auth	Purpose
POST	/onboarding	User	Rider profile and vehicle
POST	/documents	Rider	Attach licence and vehicle documents
GET	/me	Rider	Status, zone, rating, today's stats
PATCH	/availability	Rider	Go online or offline
GET	/offers	Rider	Pending delivery offers
POST	/offers/{id}/accept	Rider	Accept offer

Method	Path	Auth	Purpose
POST	/offers/{id}/reject	Rider	Reject offer
GET	/deliveries/active	Rider	Current delivery
GET	/deliveries/{id}	Rider	Delivery detail
POST	/deliveries/{id}/arrived-pickup	Rider	Arrived at store
POST	/deliveries/{id}/picked-up	Rider	Picked up
POST	/deliveries/{id}/arrived-dropoff	Rider	Arrived at customer
POST	/deliveries/{id}/delivered	Rider	Delivered with proof file and COD amount
POST	/deliveries/{id}/failed	Rider	Failed with reason
POST	/location	Rider	Batched location fallback when the socket is down
GET	/earnings?period=	Rider	Earnings and tips
GET	/cod	Rider	COD collected and owed
POST	/cod/deposits	Rider	Record a COD cash deposit
POST	/withdrawals	Step-up	Withdraw earnings to wallet or bank

12. Endpoints — Tenant Admin API and Platform Console API

The admin-bff checks the listed permission on every call. Endpoints marked MC create an approval request and return 202 instead of applying the change.

Tenant admin API (/admin/v1)

Area	Method	Path	Permission	MC
Session	GET	/me	any	
Session	GET	/search?q=	any (results filtered by permission)	

Area	Method	Path	Permission	MC
Dashboard	GET	/dashboard/overview	dashboard.view	
Dashboard	GET	/dashboard/timeseries?metric=&from=&to=	dashboard.view	
Users	GET	/users	users.view	
Users	GET	/users/{id}	users.view	
Users	GET	/users/{id}/wallets , /users/{id}/ledger , /users/{id}/devices , /users/{id}/logins	users.view	
Users	POST	/users/{id}/reveal (field, reason)	users.pii.reveal	
Users	GET, POST	/users/{id}/notes	users.edit	
Users	POST	/users/{id}/freeze , /users/{id}/unfreeze , /users/{id}/block	users.status	MC
Users	POST	/users/{id}/devices/{deviceId}/revoke	users.edit	
Users	POST	/users/{id}/limits (override)	limits.edit	MC
Wallets	POST	/wallets/{id}/adjustments	ledger.adjust	MC
KYC	GET	/kyc/cases?status=&assignee=	kyc.view	
KYC	GET	/kyc/cases/{id}	kyc.view	
KYC	GET	/kyc/documents/{id}/url	kyc.view (logged)	
KYC	POST	/kyc/cases/{id}/assign	kyc.assign	
KYC	POST	/kyc/cases/{id}/decision	kyc.decide	MC when flagged
KYC	GET, POST, PATCH	/kyc/reason-codes	kyc.config	
KYB	GET	/businesses , /businesses/{id}	kyb.view	
KYB	POST	/businesses/{id}/decision	kyb.decide	MC
Transactions	GET	/transactions , /transactions/{id}	txn.view	
Transactions	POST	/transactions/{id}/hold , /transactions/{id}/release	txn.hold	
Transactions	POST	/transactions/{id}/reverse , /transactions/{id}/refund	txn.reverse	MC
Transactions	GET	/disputes	txn.view	
Transactions	POST	/disputes/{id}/decision	txn.dispute	MC above threshold
Compliance	GET	/aml/alerts	aml.view	
Compliance	POST	/aml/alerts/{id}/triage	aml.triage	
Compliance	GET, POST	/aml/cases , /aml/cases/{id}	aml.view, aml.case	
Compliance	POST	/aml/cases/{id}/notes	aml.case	

Area	Method	Path	Permission	MC
Compliance	POST	/aml/cases/{id}/decision	aml.decide	MC
Compliance	POST	/aml/cases/{id}/str	aml.str	MC
Compliance	GET, POST	/risk/rules	risk.view, risk.edit	
Compliance	PATCH	/risk/rules/{id}	risk.edit	
Compliance	POST	/risk/rules/{id}/simulate	risk.edit	
Compliance	POST	/risk/rules/{id}/activate , /risk/rules/{id}/disable	risk.activate	MC
Compliance	GET, POST, DELETE	/blocklists	risk.blocklist	
Compliance	GET	/screening/results , /sanctions/status	aml.view	
Compliance	POST	/screening/results/{id}/decision	aml.decide	
Cards	GET	/cards , /cards/{id} , /card-authorizations	cards.view	
Cards	POST	/cards/{id}/block	cards.block	
Cards	GET	/card-disputes	cards.view	
Cards	POST	/card-disputes/{id}/submit	cards.dispute	
Cards	GET, POST, PATCH	/card-programs	cards.config	MC
Finance	GET	/treasury/positions	finance.view	
Finance	GET	/bank-transfers	finance.view	
Finance	POST	/bank-transfers/{id}/retry	finance.ops	
Finance	GET	/recon/runs , /recon/exceptions , /statement-lines?matchStatus=	finance.view	
Finance	POST	/statement-lines/{id}/match	finance.recon	
Finance	POST	/recon/exceptions/{id}/resolve	finance.recon	MC
Reports	GET	/reports/definitions	reports.view	
Reports	POST	/reports/runs	reports.run	
Reports	GET	/reports/runs/{id}	reports.view	
Reports	GET, POST, DELETE	/reports/schedules	reports.schedule	
Marketplace	GET, POST, PATCH	/merchants , /merchants/{id}	merchants.view, merchants.edit	
Marketplace	PATCH	/merchants/{id}/commercial	merchants.commercial	MC
Marketplace	POST	/merchants/{id}/suspend , /merchants/{id}/reactivate	merchants.status	MC

Area	Method	Path	Permission	MC
Marketplace	GET, POST, PATCH, DELETE	/categories	catalog.edit	
Marketplace	GET	/products?status=PENDING_REVIEW	catalog.view	
Marketplace	POST	/products/{id}/moderate	catalog.moderate	
Marketplace	GET	/orders, /orders/{id}	orders.view	
Marketplace	POST	/orders/{id}/cancel	orders.cancel	
Marketplace	GET, POST, PATCH	/promotions	promotions.edit	MC above budget limit
Riders	GET	/riders, /riders/{id}	riders.view	
Riders	POST	/riders/{id}/approve, /riders/{id}/suspend	riders.status	
Riders	GET	/deliveries	riders.view	
Riders	POST	/deliveries/{id}/reassign	riders.dispatch	
Riders	GET, POST, PATCH	/zones	riders.zones	
Payouts	GET	/payout-batches, /payouts	payouts.view	
Payouts	POST	/payout-batches/{id}/hold, /payouts/{id}/retry	payouts.ops	MC
Limits & fees	GET, POST, PATCH	/limit-rules	limits.edit	MC
Limits & fees	GET, POST, PATCH	/fee-rules	fees.edit	MC
App control	GET, POST, PATCH	/flags	app.flags	
App control	POST	/flags/{id}/toggle	app.flags	MC for kill switches
App control	GET, POST	/segments	app.segments	
App control	GET	/screens	app.sdui	
App control	POST, PATCH	/screens/{id}/versions, /screens/{id}/versions/{v}	app.sdui	
App control	POST	/screens/{id}/versions/{v}/publish, /screens/{id}/rollback	app.sdui.publish	MC
App control	GET, POST	/app-versions, /maintenance-windows	app.release	MC for force update
App control	GET, POST, PATCH	/notification-templates	app.content	
App control	GET, POST, PATCH	/campaigns	app.campaigns	

Area	Method	Path	Permission	MC
App control	POST	/campaigns/{id}/schedule	app.campaigns	MC
App control	GET, POST, PATCH	/billers	app.billers	
Support	GET	/tickets , /tickets/{id}	support.view	
Support	PATCH	/tickets/{id} (status, assignee, priority)	support.edit	
Support	POST	/tickets/{id}/messages	support.reply	
Support	GET, POST, PATCH	/macros	support.config	
Admin	GET, POST, PATCH	/admin-users	admin.users	
Admin	POST	/admin-users/{id}/disable	admin.users	
Admin	GET, POST, PATCH	/roles	admin.roles	MC
Admin	GET	/permissions	admin.roles	
Admin	GET	/approvals , /approvals/{id}	approvals.view	
Admin	POST	/approvals/{id}/approve , /approvals/{id}/reject	approvals.decide (not the maker)	
Admin	GET, PATCH	/approval-policies	admin.policies	MC
Admin	GET	/audit-logs	audit.view	
Admin	GET, POST, DELETE	/api-keys	admin.integrations	
Admin	GET, PATCH	/settings	admin.settings	

Platform console API (/console/v1)

Method	Path	Purpose
GET, POST	/tenants	List and create tenants
GET, PATCH	/tenants/{id}	Tenant detail and basic fields
POST	/tenants/{id}/suspend , /tenants/{id}/reactivate	Change tenant status
GET, POST	/tenants/{id}/profiles	Profile versions; create a draft

Method	Path	Purpose
POST	/tenants/{id}/profiles/{version}/activate	Activate a profile version (maker-checker)
GET	/adapters/catalog	Available adapters and their config schema
GET, POST, PATCH, DELETE	/tenants/{id}/adapters	Assign adapters and secret references
POST	/tenants/{id}/adapters/{adapterId}/test	Connectivity test against the partner sandbox
GET, POST, DELETE	/tenants/{id}/domains	Domains and TLS status
GET, POST	/tenants/{id}/apps	App flavors per platform
POST	/tenants/{id}/apps/{appId}/builds	Trigger a white-label app build in CI
GET	/tenants/{id}/deployments	Deployments and versions
POST	/tenants/{id}/deployments	Deploy a version to an environment
POST	/deployments/{id}/rollback	Roll back a deployment
POST	/tenants/{id}/admins	Invite the tenant's first Super Admin
GET	/health/tenants	Cross-tenant health and SLOs
GET	/usage?month=	Usage per tenant for billing
GET, POST, PATCH	/platform-admins	Platform staff
GET	/audit-logs	Platform audit log

13. Endpoints — Partner Webhooks and Internal gRPC

Partners call these webhooks; services call each other through the gRPC contracts below. Webhooks verify the signature (or mTLS) first and store the raw payload. Except for card authorization, they acknowledge within 200 ms and process asynchronously.

Inbound partner webhooks

Method	Path	Handler	Notes
POST	<code>/webhooks/card-issuer/{provider}/authorizations</code>	cards-auth (Go)	Synchronous approve/decline; p99 < 500 ms, hard budget 1.5 s; idempotent on processor auth ID
POST	<code>/webhooks/card-issuer/{provider}/events</code>	cards-service	Clearing, reversals, card status, token lifecycle, dispute updates
POST	<code>/webhooks/bank/{provider}</code>	banking-service	Inbound credits, payout results, returns, virtual account events
POST	<code>/webhooks/kyc/{provider}</code>	kyc-service	Check results, applicant status
POST	<code>/webhooks/acquirer/{provider}</code>	payments-service	Card top-up 3DS results, captures, refunds, chargebacks
POST	<code>/webhooks/biller/{provider}</code>	bills-service	Asynchronous bill payment status
POST	<code>/webhooks/messaging/{provider}</code>	notification-service	SMS and email delivery receipts

The tenant is resolved from the webhook domain or a tenant-scoped path secret. Files that arrive by SFTP (bank statements, pain.002, card clearing and settlement files) are picked up by scheduled BullMQ jobs, not webhooks.

Internal gRPC services

Service	RPCs
<code>ledger.v1.LedgerService</code>	CreateAccount, PostEntry, ReverseEntry, CreateHold, CaptureHold, ReleaseHold, GetBalance, BatchGetBalances, ListPostings, GetEntry
<code>wallet.v1.WalletService</code>	GetWallet, GetWalletByOwner, ReserveLimit, CommitLimit, ReleaseLimit, SetWalletStatus
<code>risk.v1.RiskService</code>	Evaluate (stage, subject, amount, device, counterparty) → ALLOW, CHALLENGE, HOLD or BLOCK with matched rules
<code>auth.v1.AuthService</code>	GetUser, BatchGetUsers, VerifyStepUpToken, SetUserStatus, RevokeSessions
<code>kyc.v1.KycService</code>	GetTier, GetProfile, RequireTier
<code>tenant.v1.TenantService</code>	GetProfile, ResolveAdapter (partnerType), ResolveDomain, ListActiveTenants
<code>platform.v1.PlatformService</code>	EvaluateFlags (user context), IsServiceEnabled, GetScreen
<code>payments.v1.PaymentsService</code>	CreatePayment, GetPayment, RefundPayment, CapturePayment
<code>banking.v1.BankingService</code>	VerifyAccount, InitiatePayout, GetTransfer
<code>cards.v1.CardsService</code>	GetCard, BlockCard, GetAuthorization
<code>delivery.v1.DeliveryService</code>	CreateDelivery, CancelDelivery, GetDelivery
<code>files.v1.FileService</code>	CreateUploadUrl, GetDownloadUrl, GetFileMeta

Every RPC carries `tenant-id`, `request-id` and `traceparent` metadata. Deadlines are set by the caller (default 2 s; ledger and risk 300 ms). Retries apply only to idempotent RPCs or those carrying an idempotency key.

14. Realtime — WebSocket Gateway

All live updates go through one realtime-gateway service running Socket.IO with the Redis adapter. Domain services never hold sockets. They publish Kafka events; the gateway consumes the relevant topics and emits them to rooms. Rider location is the one exception: it flows client → gateway → Redis → subscribers.

Connection

- URL: `wss://<tenant api domain>/socket` with WebSocket-only transport (no long-polling, so no sticky sessions needed).
- Handshake: `auth: { token: <JWT> }`; the gateway verifies the token, tenant and device, then joins the socket to its rooms. Admin sockets use the admin-bff session token.
- Heartbeat every 25 s; on token expiry the server emits `session.expired` and the client reconnects with a fresh token.
- Events carry `eventId` and a resource `version`; after reconnecting, the client refetches over REST anything newer than the last version it saw. Sockets are a fast path, never the only source of truth.
- Limits: 5 sockets per user, rider location at most one message every 3 s, payloads under 4 KB.

Namespaces and rooms

Namespace	Who	Rooms joined
<code>/app</code>	Consumers	<code>t:{tid}:user:{userId}</code> ; <code>t:{tid}:order:{orderId}</code> on subscribe
<code>/merchant</code>	Merchant staff	<code>t:{tid}:merchant:{merchantId}</code> , <code>t:{tid}:store:{storeId}</code> for each allowed store
<code>/rider</code>	Riders	<code>t:{tid}:rider:{riderId}</code> , <code>t:{tid}:zone:{zoneId}</code>
<code>/admin</code>	Tenant staff	<code>t:{tid}:admin</code> , plus <code>t:{tid}:admin:{module}</code> for each module the staff member can view

Server → client events

Namespace	Event	Payload	Source topic
/app	wallet.balance_updated	walletId, available, posted, currency, version	ledger.entries
/app	payment.status_changed	paymentId, status, amount, counterparty	payments.payments
/app	payment_request.received	requestId, from, amount, note	payments.requests
/app	card.transaction	cardId, amount, merchant, decision	cards.authorizations
/app	order.status_changed	orderId, status, eta	marketplace.orders
/app	delivery.location	orderId, lat, lng, heading, eta	Redis rider:loc channel
/app	kyc.status_changed	tier, status, requiredActions	kyc.applications
/app	notification.new	id, title, body, deeplink	notify.inbox
/app	config.changed	scope (flags, screen:{key})	platform.config
/app	session.revoked	reason	identity.sessions
/merchant	order.new , order.cancelled	order summary	marketplace.orders
/merchant	payment.received	paymentId, amount, storeId, payer masked	payments.payments
/merchant	rider.assigned , rider.arriving	orderId, rider name, eta	delivery.deliveries
/merchant	payout.status_changed	payoutId, status, amount	payouts.payouts

Namespace	Event	Payload	Source topic
/rider	offer.new	offerId, pickup, dropoff, fee, expiresAt	delivery.offers
/rider	offer.expired, delivery.cancelled	offerId or deliveryId	delivery.offers, delivery.deliveries
/rider	order.ready	deliveryId	marketplace.orders
/admin	txn.feed	masked transaction summary (sampled for high volume)	payments.payments
/admin	kyc.case_created	caseId, priority, slaDueAt	kyc.cases
/admin	aml.alert_created	alertId, severity, rule	risk.alerts
/admin	approval.requested, approval.decided	approvalId, actionType, maker	admin.approvals
/admin	riders.positions	riderId, lat, lng, status (per zone)	Redis rider:loc channel
/admin	system.alert	severity, message	ops.alerts

Client → server events

Namespace	Event	Payload	Ack
/app	order.subscribe, order.unsubscribe	orderId (ownership checked)	ok or error
/rider	rider.location	lat, lng, heading, speed, accuracy, recordedAt	ok; the gateway writes Redis GEO, publishes rider:loc, and batches to Kafka delivery.locations

Namespace	Event	Payload	Ack
/rider	rider.availability	ONLINE or OFFLINE	new status
/admin	zone.subscribe	zoneld	ok

15. Messaging — Kafka, Temporal, Job Queues, Notifications

Each kind of asynchronous work gets its own tool. Kafka carries facts that other services react to. Temporal runs multi-step money workflows that must survive crashes. BullMQ runs short background jobs. Everything is idempotent and keyed by tenant.

Kafka topics

Cluster settings: replication 3, `min.insync.replicas` 2, producers with `acks=all` and idempotence on, Avro schemas in Schema Registry with backward compatibility. The message key is `tenantId:ownerId` for per-owner ordering. Each topic has a retry topic (`<topic>.retry.1m`, `<topic>.retry.10m`) and a dead-letter topic (`<topic>.dlq`) with alerting.

Topic	Producer	Main consumers	Partitions	Retention
<code>tenancy.tenants</code>	tenant-service	all services (cache invalidation)	6	30 days, compacted
<code>identity.users</code> , <code>identity.sessions</code>	auth-service	risk, notification, realtime, reporting	24	7 days
<code>kyc.applications</code> , <code>kyc.cases</code>	kyc-service	wallet (limits), cards, realtime, notification, audit	24	7 days
<code>risk.decisions</code> , <code>risk.alerts</code>	risk-service	admin realtime, reporting	24	7 days
<code>ledger.entries</code> , <code>ledger.holds</code>	ledger-service	realtime (balances), risk (post-txn), reporting, notification	48	14 days

Topic	Producer	Main consumers	Partitions	Retention
<code>payments.payments</code> , <code>payments.requests</code>	payments-service	risk, notification, realtime, loyalty, reporting	48	14 days
<code>banking.transfers</code> , <code>banking.recon</code>	banking-service	payments, payouts, finance alerts, reporting	24	14 days
<code>cards.cards</code> , <code>cards.authorizations</code> , <code>cards.transactions</code>	cards-service	risk, notification, realtime, reporting	48	14 days
<code>bills.payments</code>	bills-service	notification, reporting	12	7 days
<code>marketplace.orders</code> , <code>marketplace.products</code>	marketplace-service	payments, delivery, notification, realtime, search indexer	24	7 days
<code>delivery.deliveries</code> , <code>delivery.offers</code>	delivery-service	marketplace, notification, realtime	24	7 days
<code>delivery.locations</code>	realtime-gateway (batched)	delivery (history), reporting	24	3 days
<code>payouts.payouts</code>	payouts-service	notification, realtime, reporting	12	14 days
<code>platform.config</code>	platform-service	realtime (<code>config.changed</code>), gateway caches	6	7 days
<code>admin.approvals</code> , <code>admin.actions</code>	admin-bff	audit-service, realtime, target services	12	30 days
<code>notify.requests</code>	any service	notification-service	24	3 days
<code>ops.alerts</code>	recon jobs, monitors	realtime <code>/admin</code> , on-call paging	3	7 days

Temporal workflows

Workflow	Owner	Steps
CardTopUpWorkflow	payments	Create acquirer payment → wait for 3DS/webhook (timeout) → post ledger entry → notify
BankWithdrawalWorkflow	payments + banking	Hold → risk → payout via BankAdapter → poll or await webhook → capture or release → notify
InboundCreditWorkflow	banking	Match credit → post to wallet or suspense → notify or raise exception
BillPaymentWorkflow	bills	Hold → biller pay → await status → capture or release → receipt
OrderPaymentWorkflow	marketplace	Hold at checkout → await delivered or cancelled (days) → capture and split, or release
PayoutBatchWorkflow	payouts	Build batch → net reserves and disputes → bank payouts → reconcile → statements
KycReviewWorkflow	kyc	Provider checks → decision engine → manual case with SLA timer → tier change
CardIssuanceWorkflow	cards	Eligibility → CardIssuerAdapter create → physical shipment tracking → activation
CardDisputeWorkflow	cards	Submit → provisional credit → await outcome before scheme deadline → final posting
ApprovalExecutionWorkflow	admin-bff	Await checker decision (expiry) → execute the original call with its idempotency key
TenantOnboardingWorkflow	tenant-service	Create profile → provision infra → seed ledger accounts and roles → adapter tests → activate

Workflows run in one Temporal namespace per deployment, carry `tenantId` as a search attribute, and call services only through activities that pass idempotency keys.

BullMQ queues

Queue	Type	Purpose
<code>notify.push</code> , <code>notify.sms</code> , <code>notify.email</code>	On demand	Send through the tenant's MessagingAdapter with retries
<code>webhooks.merchant</code>	On demand	Deliver signed merchant webhooks with backoff for 72 h
<code>reports.run</code>	On demand + scheduled	Generate reports and exports to S3
<code>statements.generate</code>	On demand + monthly	Wallet and merchant statements (PDF)
<code>files.scan</code>	On demand	Virus scan uploaded files before use
<code>sftp.ingest</code>	Repeatable (per bank, per processor)	Pull statements, pain.002 and clearing files
<code>recon.daily</code>	Repeatable (daily)	Internal, bank, card and safeguarding reconciliation
<code>holds.expire</code>	Repeatable (every 5 min)	Release expired holds
<code>sanctions.sync</code>	Repeatable (daily)	Refresh sanctions lists and re-screen
<code>segments.refresh</code>	Repeatable (hourly)	Recompute segment membership
<code>campaigns.send</code>	Scheduled	Fan out approved campaigns in batches

Notification pipeline

1. A service publishes a domain event (for example `payments.payments` with status COMPLETED) or an explicit `notify.requests` message.

- notification-service maps the event type to the tenant's templates per channel and the user's locale.
- It applies preferences and quiet hours; security and transaction messages ignore marketing opt-outs.
- It writes a `notifications` row (deduplicated by event ID and channel) and enqueues the channel job.
- The channel worker sends through the tenant's MessagingAdapter (FCM for push, the local SMS provider, email provider).
- Delivery receipts update the row; in-app messages also emit `notification.new` on the socket.

OTP messages skip the queue: auth-service calls the MessagingAdapter directly, with provider failover, to keep delivery under a few seconds.

16. Caching, Observability, Testing and CI/CD

Balances are never served from cache. Everything else that is read-heavy and tenant-scoped is cached in Redis, and every release passes the same automated gates.

Redis keys

Key pattern	Content	TTL
<code>t:{tid}:profile</code>	Tenant profile	5 min + invalidated on <code>tenancy.tenants</code>
<code>t:{tid}:flags</code>	Compiled feature flags	60 s + invalidated on <code>platform.config</code>
<code>t:{tid}:screen:{key}:{segment}</code>	Resolved SDUI JSON + ETag	5 min + invalidated on publish
<code>t:{tid}:idem:{key}</code>	In-flight idempotency lock	60 s
<code>t:{tid}:rl:{scope}:{id}</code>	Rate-limit sliding window	window length
<code>t:{tid}:vel:{userId}:{service}</code>	Risk velocity counters	1 h / 24 h

Key pattern	Content	TTL
<code>t:{tid}:otp:{phoneHash}</code>	OTP attempt counter	15 min
<code>t:{tid}:riders:geo</code>	GEO set of online riders	live, removed when offline
<code>t:{tid}:stepup:{tokenId}</code>	One-time step-up token	2 min

Observability

- OpenTelemetry SDK in every service; traces to Tempo, metrics to Prometheus, logs (JSON, PII-masked) to Loki; Grafana dashboards per service and per tenant.
- Every log line and span carries `tenant_id`, `request_id`, `user_id` (hashed) and `payment_id` when present.
- Business SLIs: payment success rate per flow and rail, card approval rate and auth latency, KYC auto-approval rate, OTP delivery time, recon exceptions, socket connections.
- Alerts: SLO burn rate, ledger imbalance (any non-zero is P1), DLQ growth, webhook backlog, partner error rate per adapter per tenant.

Testing

Level	Tooling	Required coverage
Unit	Vitest (TS), <code>go test</code> (Go)	Domain logic, fee and limit calculation, state machines
Integration	Testcontainers (Postgres, Redis, Kafka, Temporal)	Repositories, RLS isolation between two tenants, outbox, consumers
Ledger properties	fast-check / Go fuzzing	Random flows always balance; balances equal the sum of postings
Contract	Pact (REST), buf breaking checks (gRPC), AsyncAPI schema checks	Every consumer-provider pair and every adapter
Adapter	Partner simulators	Timeouts, duplicates, out-of-order webhooks, partial failures

Level	Tooling	Required coverage
End to end	Playwright (admin web), Patrol or Maestro (Flutter)	Onboarding, P2P, top-up, card auth, order to delivery, maker-checker
Load	k6	3× forecast peak; card authorization p99 under 500 ms
Security	Semgrep, Trivy, OWASP ZAP, annual penetration test	No high findings at release

CI/CD

1. Pull request: lint, type-check, unit and integration tests for affected Nx projects, contract checks, SAST and dependency scan.
2. Merge to main: build signed container images (one per service), generate OpenAPI and AsyncAPI clients, publish to the registry.
3. Deploy to dev and staging automatically through Argo CD; run end-to-end and adapter simulator suites.
4. Release: tag a platform version; Argo CD rolls it out to tenant stacks in waves (internal → pilot tenants → all) with canary analysis and automatic rollback.
5. Mobile: Codemagic builds one flavor per tenant from the same commit and uploads to each tenant's store accounts.